Orchestrator: A Deterministic Execution Engine for Device Interaction Runs

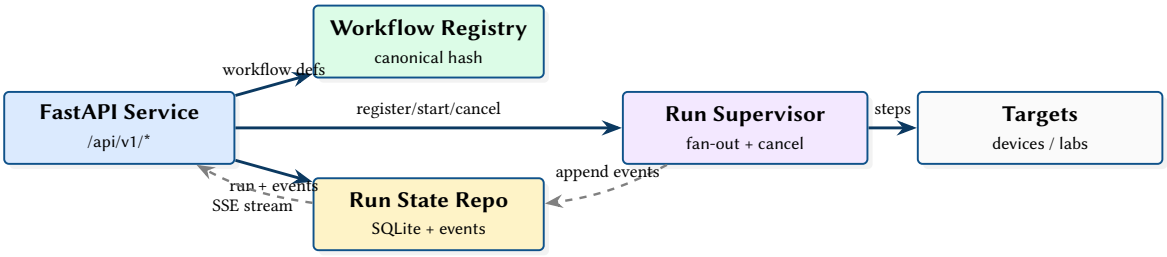
Run Coordination, Durable State, and Streaming Observability for Network Automation

Simon Knight

Network Automation Ecosystem

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. This report documents the Orchestrator service: an execution engine for reliable, replayable device interaction runs. The system offers a small workflow DSL (v1), schema-validated registration, per-run durability using SQLite, bounded concurrency, cancellation, and a server-sent event (SSE) stream of structured run events. The goal is deterministic run coordination with clean integration boundaries (device interaction libraries handle transport/parsing; the orchestrator owns control-plane semantics and state). We describe the design, core data models, APIs, and execution semantics, and present a set of diagrams that capture the system's state machine, concurrency controls, and event flow.

Contents

List of Design Decisions & Rules

1 Problem Statement and Scope

Network automation in real labs and testbeds requires more than a library of device operations. The missing component is a reliable coordinator that executes the same workflow across a target set with deterministic semantics, strong observability, and durable run history.

This repository implements the Orchestrator engine as an API-first service (FastAPI) and a small SDK for authoring and parsing workflow definitions. It is intentionally not a full Tower/AWX clone. The v1 scope is device-focused execution semantics and a control plane that can be embedded in CLI/Workbench/CI.

The authoritative requirements and sequencing are captured in the in-repo planning documents (`.planning/PROJECT.md`, `.planning/REQUIREMENTS.md`, `.planning/ROADMAP.md`) [1–3].

Design Decision 1: API-first Service

The orchestrator exposes a stable HTTP API as its primary interface. This decouples execution semantics from any single client UX and supports embedding into Workbench, CLI, and CI pipelines.

Design Decision 2: Device Interaction Boundary

The orchestrator owns: run coordination, state persistence, concurrency controls, retries/timeouts, cancellation, and event streaming. Transport, parsing, and device capability logic are delegated to a device interaction library via adapters.

2 System Overview

At a high level, Orchestrator consists of: (i) a workflow registry, (ii) a run state repository with event log, (iii) a supervisor that executes runs across targets, and (iv) an API layer that registers workflows, starts runs, cancels runs, and streams events.

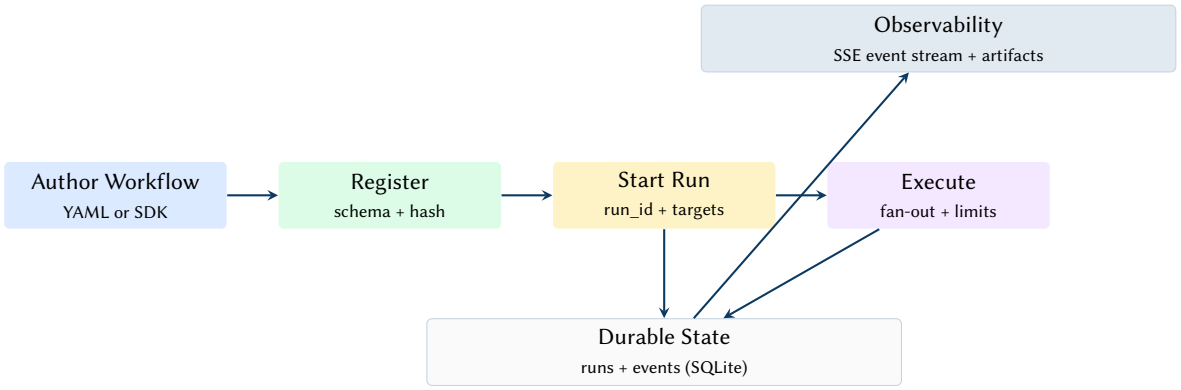


Figure 1. End-to-end lifecycle: authoring, registration, execution, and observability.

3 Workflow Definition and Validation

Insight:
A run is not "a script"; it is a durable, replayable execution record. The core output of the engine is a consistent event stream that correlates run, target, and step.

Workflows are represented as validated Pydantic models (namespace, name, DSL version, and an ordered list of steps). Each step declares an adapter, a typed inputs map, and explicit dependencies (`depends_on`).

The YAML parser is designed to avoid YAML 1.1 coercion pitfalls by enforcing YAML 1.2 "safe" parsing (`ruamel.yaml`, safe typ, explicit version). This is a pragmatic correctness guardrail.

Design Decision 3: YAML 1.2 Parsing

Workflow YAML is parsed using a YAML 1.2 safe loader to prevent YAML 1.1 boolean coercion (e.g., `OFF`, `NO`) from silently changing operator intent. This is a known class of parsing hazards in operational YAML configurations; enforcing YAML 1.2 semantics is a simple, high-leverage guardrail [4].

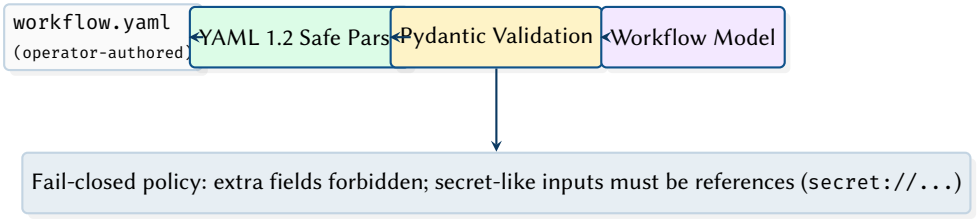


Figure 2. Workflow ingestion pipeline with validation and safety policy checks.

4 Secrets Policy

Step inputs are permitted to contain secret values only by reference. Sensitive keys (e.g., `password`, `token`, `api_key`) are rejected unless their values are a `secret://...` reference (either as a string or as a single-key map `{secret_ref: ...}`). This enforces "references only" as a structural invariant in the workflow model.

: Secret-by-Reference

Any step input whose key is sensitive MUST be expressed as a reference (`secret://...`); plaintext values are rejected at validation time.

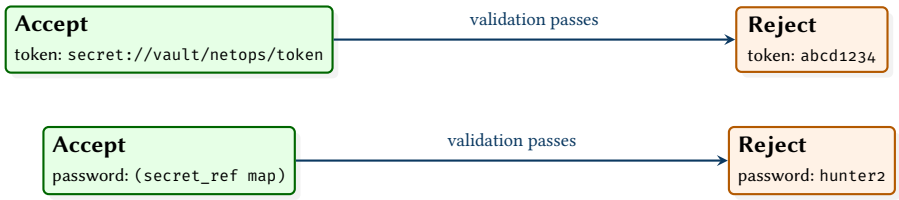


Figure 3. Secret input policy expressed as a validation invariant.

5 Run Model, Events, and Durability

Runs are persisted in SQLite with an append-only event log. A run references: the workflow identifier, the list of targets, and a run-level status. Each emitted event includes stable correlation identifiers (run, target, step) and a timestamp.

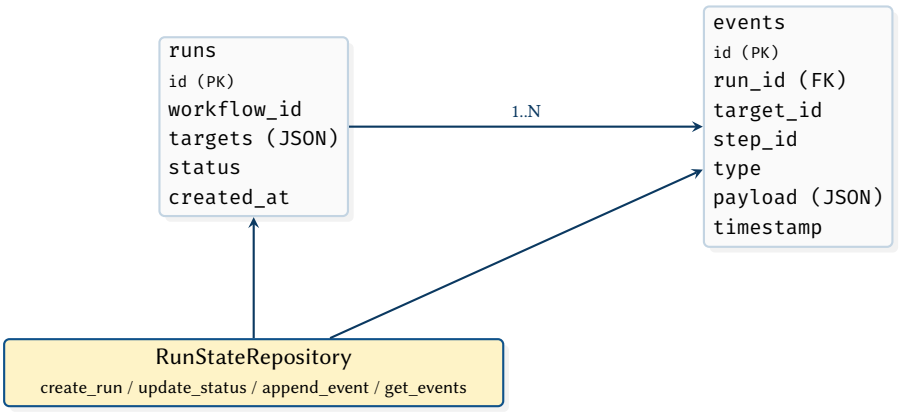


Figure 4. SQLite durability model: run header row plus correlated append-only events.

Design Decision 4: Event Log as Primary Interface

The event stream is treated as a first-class output: it enables live streaming to clients (SSE) and provides a durable audit trail for replay, debugging, and artifact correlation.

6 Execution Semantics

The supervisor executes the run by fanning out over targets and iterating the workflow steps per target. Cancellation is implemented via AnyIO cancel scopes. Concurrency is bounded by a global semaphore and optional per-target semaphores.

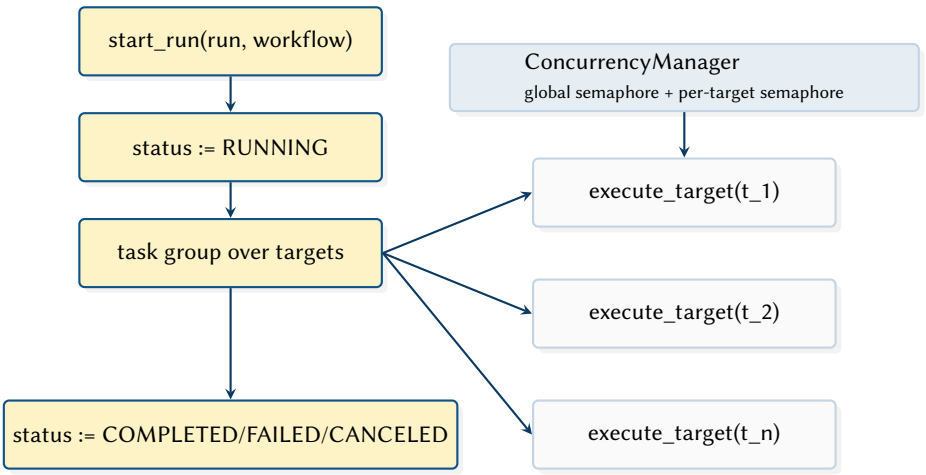


Figure 5. Run execution: fan-out across targets, step iteration per target, bounded by concurrency controls.

6.1 Concurrency as a Budgeted Scheduling Problem

Insight: *Determinism is primarily a control-plane property: bounded concurrency and explicit ordering rules reduce "action at a distance" between targets during execution.*

Linear architecture diagrams are useful as orientation, but the core of the orchestrator is a set of execution invariants: global and per-target capacity constraints, and a scheduler that admits work only when the relevant budgets allow it. This can be expressed as a token budget model.

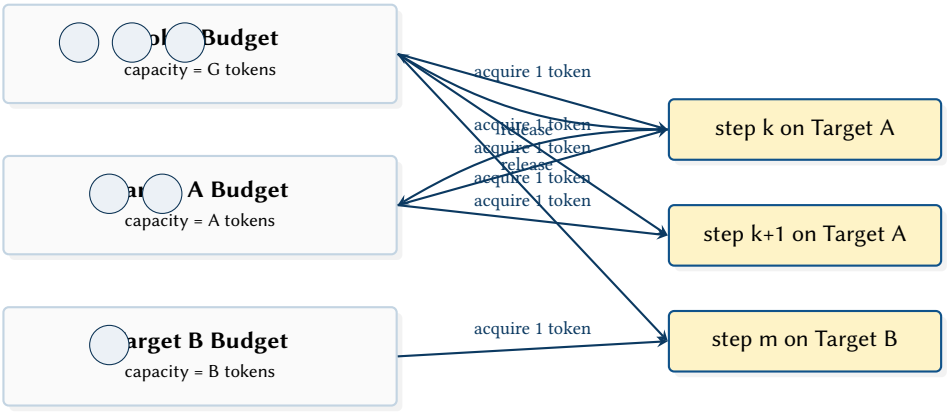


Figure 6. Concurrency as token budgets: each step admission consumes one global token and (optionally) one target token, bounding total parallelism and per-target pressure.

6.2 Cancellation Propagation and Cooperative Checkpoints

Cancellation is not simply a boolean state; it is a propagation mechanism. In AnyIO, a cancel scope is a tree: cancellation flows downward to child tasks at well-defined checkpoints (await points), making cancellation latency a function of where tasks yield.

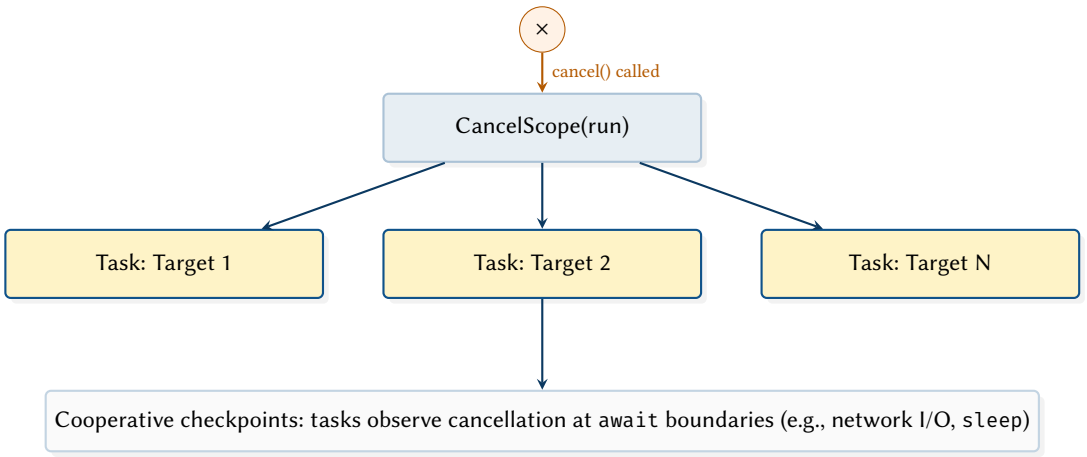


Figure 7. Cancellation propagation as a scope tree: a run-level cancel scope cancels all target tasks; responsiveness depends on cooperative await points.

6.3 Retry and Backoff as a Control System (Planned)

Although retries/backoff are not yet fully realized in v0.1.0 execution, the intended mechanism is best understood as a control loop over transient failures. The backoff policy shapes load on targets and reduces correlated failure cascades.

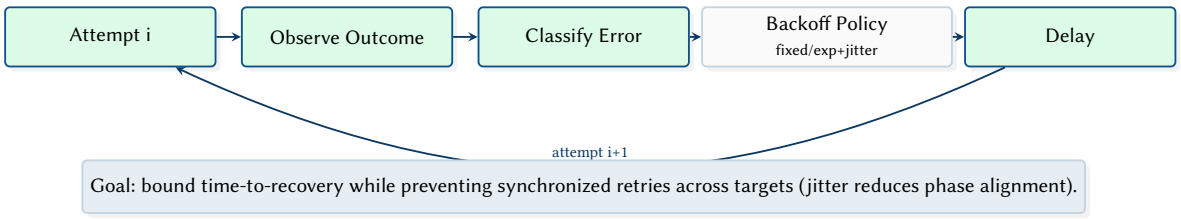


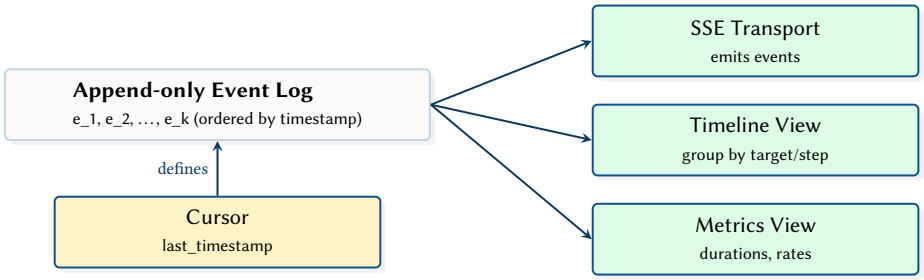
Figure 8. Retries/backoff as a control loop: outcomes are classified, a policy selects delay, and the next attempt is admitted subject to concurrency budgets.

7 Event Streaming (SSE)

Clients can subscribe to an SSE endpoint to receive a live stream of events. The event generator polls for new events after a cursor timestamp, yields them as SSE messages, and terminates once the run reaches a terminal state. This design uses a standard server-to-client streaming mechanism over HTTP (Server-Sent Events (SSE)) [5].

7.1 Event Streams as a Projection of an Append-only Log

The SSE endpoint is best understood as a projection: a client-specific view over a durable, append-only event log. This framing matters because it separates (i) the source of truth (events in SQLite) from (ii) transport (SSE) and (iii) derived views (UI timelines, metrics, summaries).



Invariant: events are append-only; consumers compute projections without mutating the source log.

Figure 9. Event streaming as projection: durable append-only events support multiple derived views; SSE is one transport for a cursor-based projection.

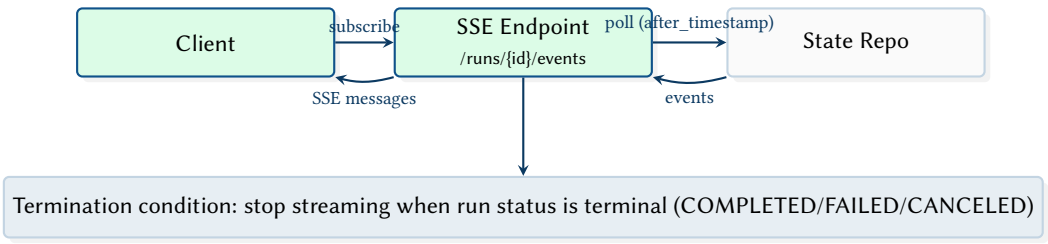


Figure 10. SSE event streaming architecture: API polls the durable event log and emits SSE messages to clients.

8 Implementation Walkthrough (Repository Mapping)

This section maps the conceptual components to concrete modules.

Module	Responsibility
`src/orchestrator/api/main.py`	FastAPI app + lifespan initialization
`src/orchestrator/api/routers/workflows.py`	Workflow registration + schema endpoint
`src/orchestrator/api/routers/runs.py`	Run creation, cancellation, and SSE event stream
`src/orchestrator/models/workflow.py`	Workflow/Step Pydantic models + secret validation policy
`src/orchestrator/engine/state.py`	`RunStateRepository` protocol + SQLite implementation
`src/orchestrator/engine/supervisor.py`	Run fan-out execution + event emission + cancellation
`src/orchestrator/engine/concurrency.py`	Global and per-target semaphore limits
`src/orchestrator/sdk/parser.py`	YAML 1.2 safe parsing into validated models

Table 1. Code-to-architecture mapping for the Orchestrator implementation.

> **Design Decision 5: Protocol-driven State**
>
> The supervisor depends on a `RunStateRepository` protocol. This keeps the execution engine decoupled from a specific persistence mechanism (SQLite in v0.1.0, but replaceable later).

9 State Machine

The current implementation uses a run-level status set with terminal states `COMPLETED`, `FAILED`, and `CANCELED`. The intended trajectory is to extend this model to per-target and per-step attempt states, while preserving the event log as the audit trail.

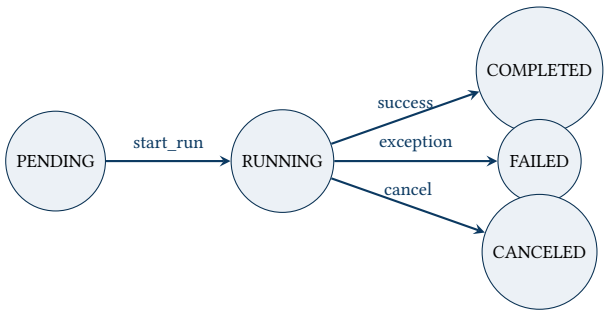


Figure 11. Run status state machine (v0.1.0).

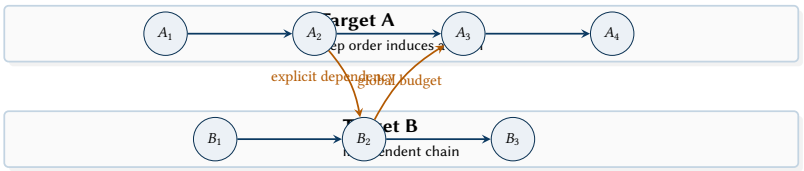
9.1 Determinism as a Partial Order Over Events

In distributed execution, "determinism" is not total ordering of all actions; it is the presence of explicit ordering constraints (a partial order) that define what must happen-before what. The orchestrator should make these constraints explicit in its event model and execution semantics.

Definition (Event set). Let E be the set of emitted run events. Each event $e \in E$ is identified by the tuple $(run\_id, target\_id, step\_id, event\_id)$ and carries a timestamp.

Definition (Happens-before). Define a relation $\prec \subseteq E \times E$ where $e_1 \prec e_2$ iff the orchestrator semantics require e_1 to occur before e_2. This includes at least: (i) *Per-target step order* edges (for a fixed target, "Step Completed" for step k precedes "Step Started" for step $k{+}1$), and (ii) *Dependency* edges induced by `depends_on`.

Definition (Determinism boundary). Two executions are equivalent for clients if they produce event streams that are linear extensions of the same partial order $(E, \prec)$. This allows concurrency-induced interleavings, while forbidding reorderings that violate required constraints.



A partial order $\prec$ defines determinism boundaries: consumers may see interleavings that preserve edges, but not a total ordering across targets.

Figure 12. Determinism as a partial order: per-target step chains plus explicit cross-target constraints (budgets/dependencies) define valid interleavings.

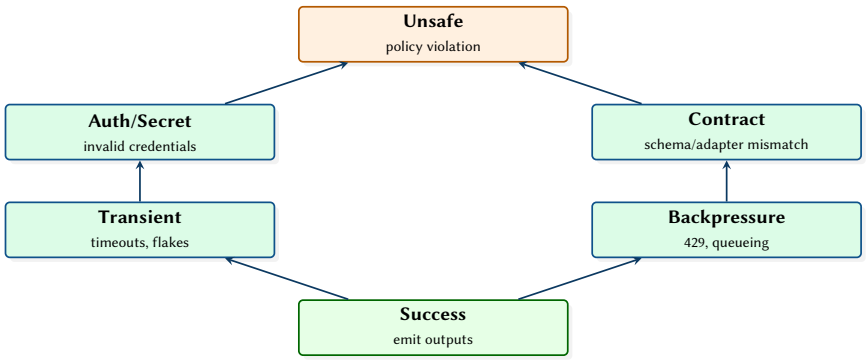
9.2 Failure Taxonomy as a Policy Lattice

Operational failures are not uniform. A robust orchestrator needs a taxonomy that maps observed errors to policy decisions (retry, skip, fail-fast, cancel, or quarantine). This can be expressed as a lattice where increasing "severity" restricts allowable actions.

Definition (Error class). Let C be a finite set of error classes (e.g., Transient, Backpressure, Auth/Secret, Contract, Unsafe). Each step attempt that fails is assigned a class $c \in C$.

Definition (Policy actions). Let A be the set of admissible actions: $A = \{\textsc{retry}, \textsc{delay}, \textsc{skip}, \textsc{failfast}, \textsc{cancel}, \textsc{quarantine}\}$. The policy returns a subset $\pi(c) \subseteq A$.

Definition (Severity order). Define a preorder $\sqsubseteq$ over classes such that $c_1 \sqsubseteq c_2$ means "c_2 is at least as severe as c_1". The policy is monotone when $c_1 \sqsubseteq c_2 \Rightarrow \pi(c_2) \subseteq \pi(c_1)$, i.e., more severe errors allow fewer actions.



Insight: *The lattice framing prevents accidental policy drift: adding a new "more severe" class cannot expand the action set without explicitly violating monotonicity.*

Policy mapping (illustrative):
Transient → retry with backoff; Backpressure → delay + jitter; Auth/Secret → fail-fast (no retry); Contract → fail-fast (developer action); Unsafe → cancel/quarantine and emit audit event.

Figure 13. Failure taxonomy lattice: severity restricts admissible actions; policies map error classes to decisions while preserving auditability.

10 Limitations and Next Steps

The current implementation is a functional skeleton that demonstrates the control-plane contracts. Several roadmap items remain to make the engine production-ready for device-first execution:

- Real device adapters (using `deviceinteraction`) instead of stubbed step execution.
- First-class step attempts with retry/backoff and timeout recording.
- Artifact storage layout, metadata, and redaction; download APIs.
- Run resumption/reconciliation after service restart.

: Preserve Event Semantics

Any expansion of execution capabilities (retries, artifacts, resumption) must preserve stable event correlation identifiers and append-only semantics.

References

[1] *Orchestrator project definition (local)*, File: .planning/PROJECT.md in /Users/simonknight/dev/orchestrator, Scope, core value, constraints, and key decisions., 2026.

[2] *Orchestrator requirements (local)*, File: .planning/REQUIREMENTS.md in /Users/simonknight/dev/orchestrator, v1 requirements and traceability to phases., 2026.

[3] *Orchestrator roadmap (local)*, File: .planning/ROADMAP.md in /Users/simonknight/dev/orchestrator, Phase sequencing and success criteria for v1., 2026.

[4] *Yaml version 1.2 (third edition)*, YAML Specification, Defines YAML 1.2 core schema; avoids YAML 1.1 boolean coercions such as ON/OFF/YES/NO., 2021.

[5] *Server-sent events*, WHATWG HTML Living Standard, EventSource interface and SSE event stream format., 2026.

List of Figures

List of Tables

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial technical report for v0.1.0 implementation